文字转语音

文字转语音
例程实验效果简介
代码解析

例程实验效果简介

这一节我们来学习一下使用K230实现将文字转为语音，并播放的功能

本节例程代码位于：【 源码汇总 / 10.Media / 06.tts_zh.py】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

我们在这里设置想要转为语音的文本 (*仅支持中文！)

```
if __name__ == "__main__":
    .............
    text = "你好亚博智能科技，我是小亚机器人"                                    # 输入中文语句 /
Input Chinese sentence
    ............
```

点击CanMV IDE左下角的运行按钮，K230会开始转换text中的文本

> 这个过程可能需要15~20秒的时间

代码解析

我们来分析这段中文TTS(文本转语音)系统的结构：

1. EncoderApp类（编码器）

- 继承自AIBase基类
- 主要功能:

```
def preprocess(self, text):
    # 对输入文本进行预处理，转换成模型可以处理的格式
    preprocess_data = aidemo.tts_zh_preprocess(self.ttszh, text)
    # 返回两个输入tensor:speaker数据和序列数据
    return [enc_speaker_input_tensor, enc_seq_input_tensor]
```

编码器的输出包含两部分:

- 音素编码向量(enc_output_0_np)
- 音素持续时间(enc_output_1_np)

2. DecoderApp类（解码器）

- 继承自AIBase基类
- 关键方法:

```
def preprocess(self, dec_input):
    # 将编码器的输出转换为decoder可处理的tensor格式
    dec_input_tensor = nn.from_numpy(dec_input)
    return [dec_input_tensor]
```

3. HifiGanApp类（声码器）

- 继承自AIBase基类
- 主要功能是将解码器输出转换为实际的音频波形
- 关键处理逻辑:

```
def run(self, dec_output_np, durition_sum):
    # 将输入分成多个子向量进行处理
    self.subvector_num = durition_sum // 100
    # 对每个子向量进行推理
    for i in range(self.subvector_num):
        results = self.inference([hifigan_input_tensor])
```

4. TTSZH类（主控制类）

- 整合了上述三个组件
- 核心执行流程:

```
def run(self, text):
    # 1. 执行编码器推理
    encoder_output_0, encoder_output_1 = self.encoder.run(text)
    # 2. 执行解码器推理
    decoder_output_0 = self.decoder.run(encoder_output_0)
    # 3. 执行声码器推理
    hifigan_output = self.hifigan.run(decoder_output_0, encoder_output_1)
    # 4. 保存并播放音频
    aidemo.save_wav(save_data, save_len, self.save_wav_file, 24000)
```

5. 音频播放功能

```
def play_audio(self):
    # 设置音频参数
    SAMPLE_RATE = 24000  # 采样率24000Hz，即每秒采样24000次 / Sample rate 24000Hz,
i.e., 24000 samples per second
    CHANNELS = 1         # 通道数，1为单声道 / Number of channels, 1 for mono
    # 使用PyAudio库进行音频播放
    output_stream = p.open(format=FORMAT, channels=CHANNELS, rate=SAMPLE_RATE,
output=True)
```

6. 主程序流程

```
if __name__ == "__main__":
    # 1. 初始化内存和退出点
    nn.shrink_memory_pool()
    # 2. 设置各种模型和配置文件路径
    encoder_kmodel_path = "/sdcard/kmodel/zh_fastspeech_1_f32.kmodel"
    # 3. 创建TTS实例并执行
    tts_zh = TTSZH(...)
    tts_zh.run(text)
```